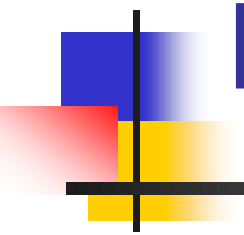
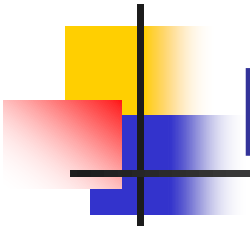


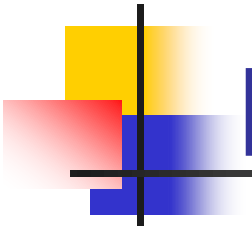
Software Development Paradigms

Software development paradigms



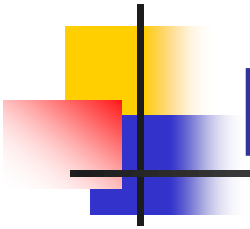
- This chapter presents basic and evolutionary software development paradigms.
- The presentation is mainly adapted from [Pre97, Pre10, Pre15, Som89, Som01, Som05, Som07, Som11, Som16].
- A software paradigm provides an abstract representation of a software process model.

Software development paradigms



- A **software process** is a set of activities (mostly carried out by software engineers) and associated results which produce a software product.
- A **software development paradigm** is a very general software process model, usually represented from an architectural perspective.

Software development paradigms

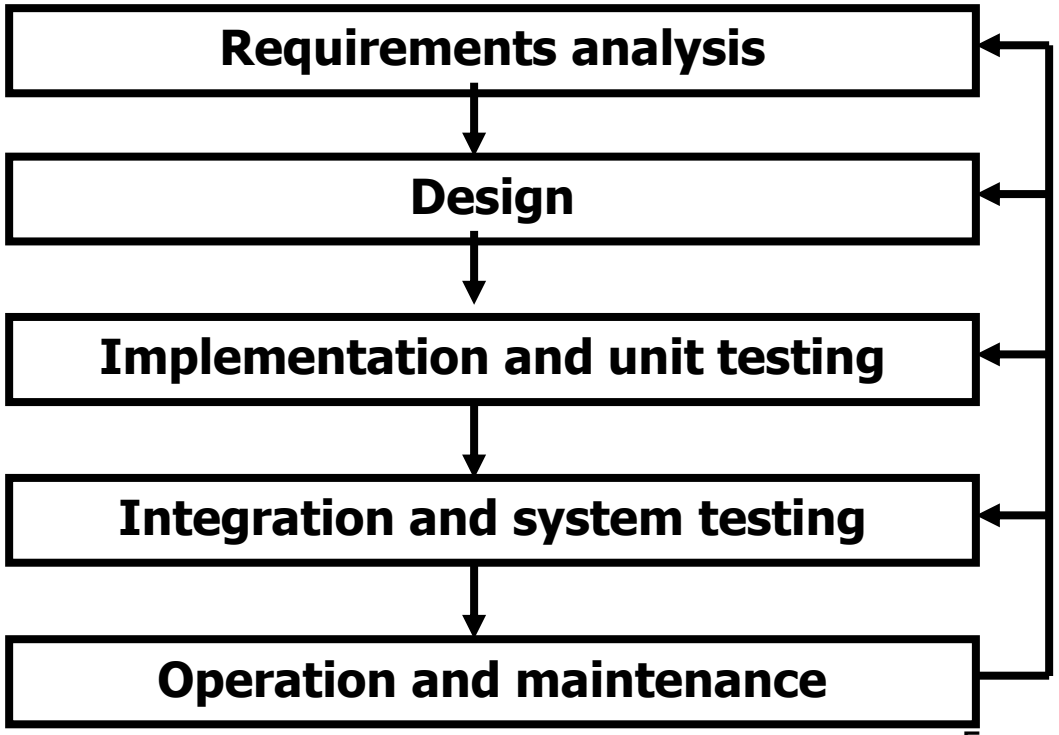


- Four **basic software paradigms** are discussed:
  - 'waterfall' (the linear model),
  - prototyping,
  - component assembly, and
  - formal methods.
- The following evolutionary software process models, which provide support for **process iteration** are also considered:
  - incremental development,
  - spiral development and
  - concurrent engineering.

Basic paradigms – waterfall

The **linear model** (or '**waterfall**') [Roy70]

- The linear model comprises the following stages:



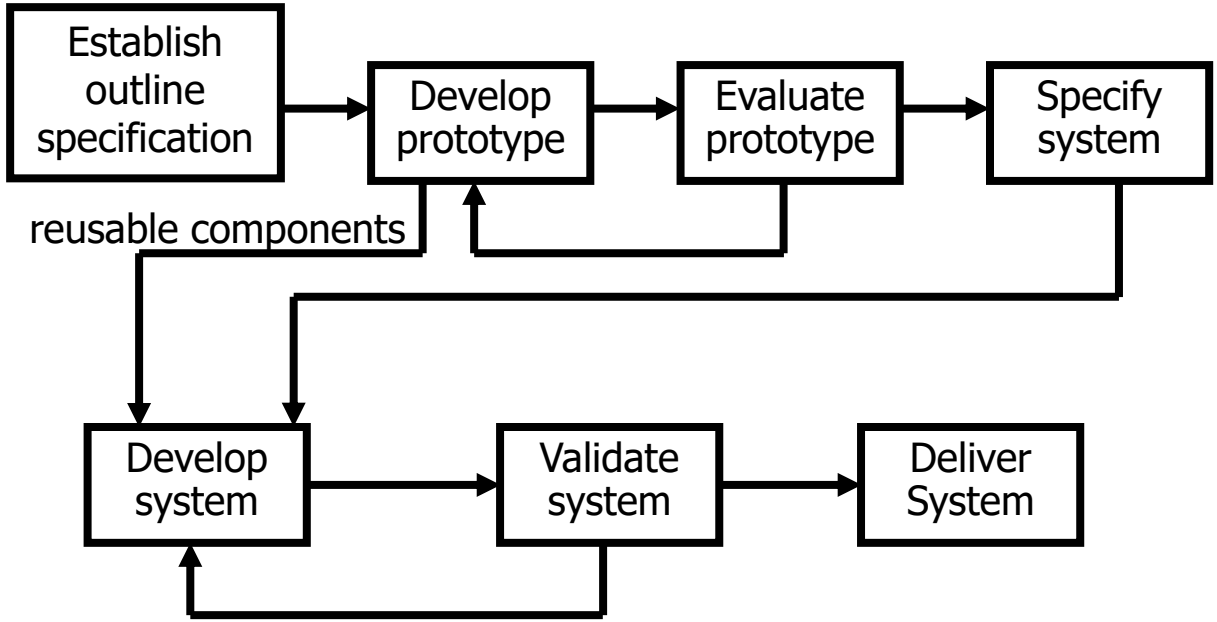
- The linear model behaves well when the requirements are clear from the beginning.
- However, in this approach it may be difficult to handle a requirements modification request (because all analysis and design decisions are taken at the beginning of the project).
- The waterfall model is employed in many small projects. In large projects it is usually employed as part of an iterative process.

Basic paradigms – prototyping

The **prototyping paradigm**

- A **prototype** is an initial system implementation which, ideally, serves only as a mechanism for identifying software requirements.

- Once the requirements are defined the prototype can be thrown-away (part of the components may be reused).
- The software system is then re-implemented to accomplish a production-quality system.



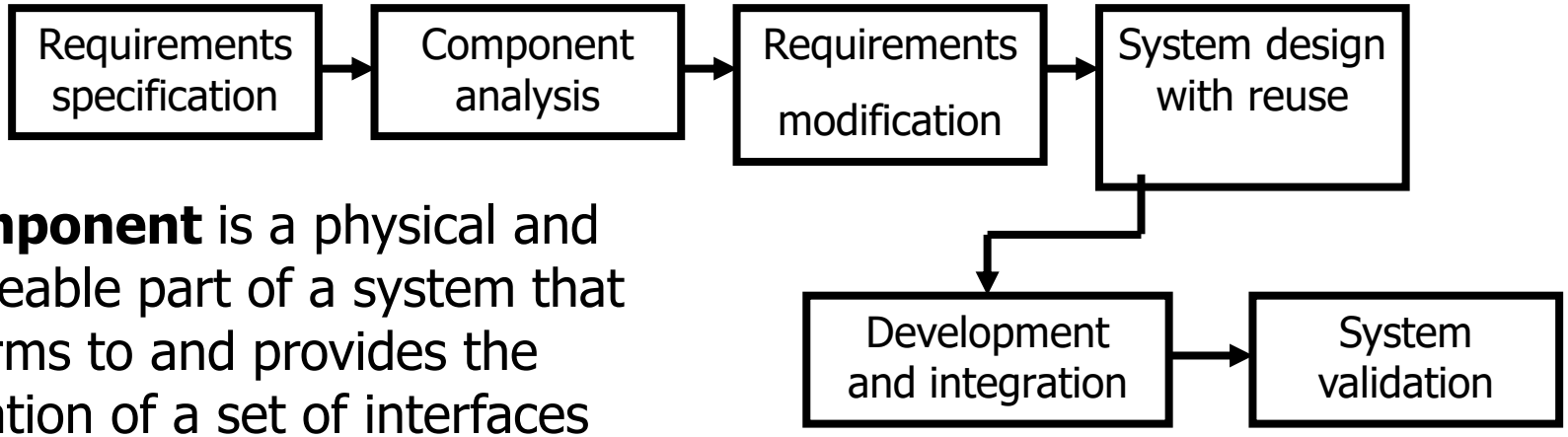
Basic paradigms – prototyping

- The prototype is only intended to demonstrate the functional (rather than the non-functional) aspects of the system.
- The prototype should be developed at a cost which is significantly lower than the cost of the final system. This can be accomplished by [Som16,Som07,Som89]:
  - relaxing non-functional requirements (speed, space requirements, etc.);
  - using a very high-level language for prototype implementation (e.g., Haskell, Prolog, Scala);
  - ignoring considerations of error action;
  - using visual development environments and automatic code generators;
  - reusing existing components, etc.

Basic paradigms – component assembly

The **component assembly model** is a reuse-oriented approach to software development

- It relies on a large base of reusable software components which can be accessed and some integrating framework for these components.



A **component** is a physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces [JBR99].

Basic paradigms – component assembly

- The stages of the reuse-oriented process [Som07,Som11]:
  - Given the **requirements specification**, during the **component analysis** phase a search is made for components to implement that specification. Usually, there is not an exact match and the components which may be used provide only some of the required functionality.
  - During **requirements modification** the requirements are analyzed and, eventually, modified to reflect the available components. Where modifications are impossible, the component analysis activity phase may be re-entered to search for alternative solutions.
  - During the **system design with reuse** phase, the framework of the system is designed by taking into account the components which are reused.
  - During the **development and integration** phase, software which can not be bought is developed and the components are integrated to create the system.

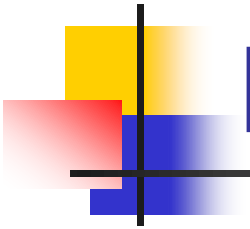
Basic paradigms – formal methods

In the **formal methods model** the software system is developed based on a mathematical (formal) specification.

- A **formal specification** provides a precise and implementation independent description of (aspects of) system behavior, and it can be seen as a mathematical 'blueprint' of the system.
- The step from a formal specification to code can be based on a formal transformation (which may be expensive), or it may be informal.
- For large systems the formal specification step is preceded by an architectural design phase, and the implementation step is followed by an integration and system testing phase.

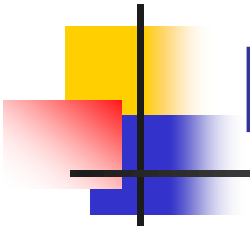
Basic paradigms – formal methods

- Formal specification enables **formal verification** of correctness which can reduce significantly the costs of software testing. In principle, formal verification of the mathematical specification can remove the need for unit testing, which is important because
- Software testing is a resource consuming phase:
  - 30%-40% in an ordinary project;
  - up to 5 times the resources of all others development activities in a project with stringent reliability requirements (cf. [Pre97]).
- The approach based on formal methods is particularly suited for the development of systems that have stringent **safety**, **reliability** or **security** requirements, such as software for aircrafts control or medical devices.
- Examples of formal methods and approaches
  - model-based specification using Z [Spi92,Jac97,Crr99]
  - Model-checking [BK08, CHVB18]
  - algebraic specification [EM85, EM90]



Process iteration

- Large sized software systems evolve over a period of time. Requirements often change as the software development proceeds.
- Therefore, software engineers need some process models designed to accommodate a product that evolves over time.
- But the evolutionary nature of software is not considered in the basic paradigms.



Process iteration

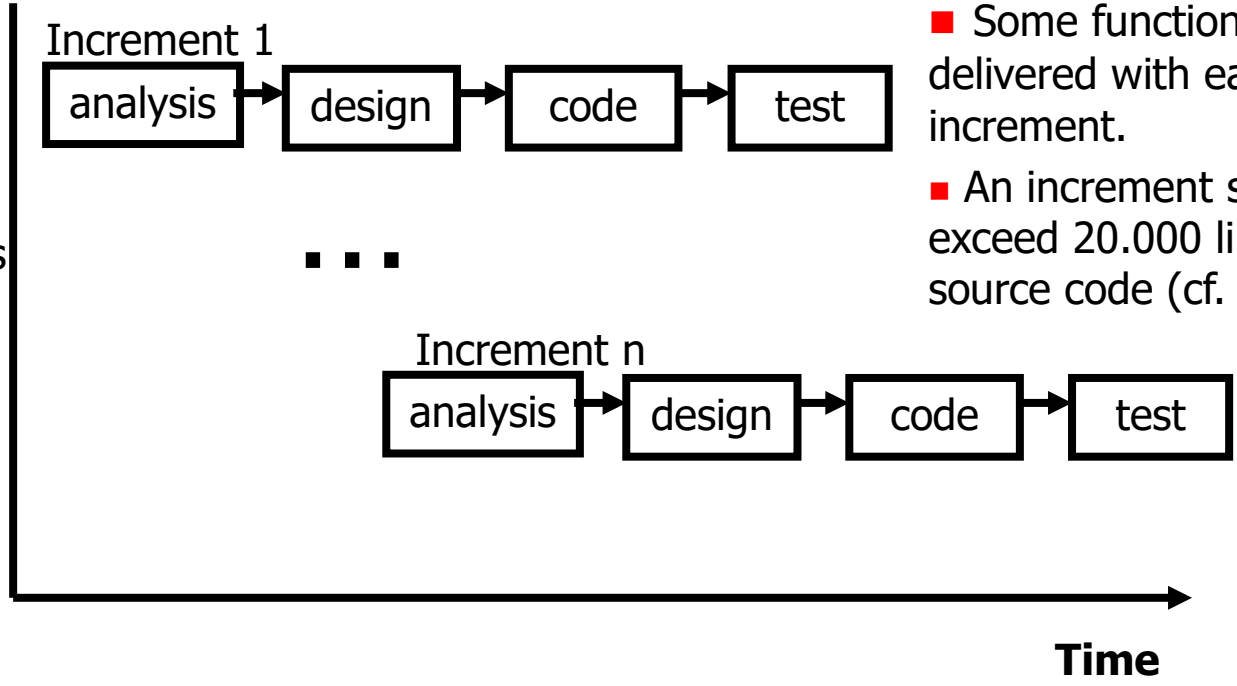
- We present three **evolutionary models**:
  - the incremental model,
  - the spiral model, and
  - concurrent engineering.
- The essence of evolutionary models:
  - **specification is developed in conjunction with software** [Som01,Som07,Som11]
  - they provide support for **process iteration**.

Process iteration – the incremental model

The **incremental model**

- Employed e.g. in RUP [Kru03,JBR99] and XP [Bec99].
- Divides the work into smaller parts, called *increments* [Pre97, Pre15]:

- Conceptually, each iteration is the basis for the next one.
- However, iterations can overlap in time (an iteration can start before the completion of the previous iteration).



- Some functionality is delivered with each increment.
- An increment should not exceed 20.000 lines of source code (cf. [Som01]).

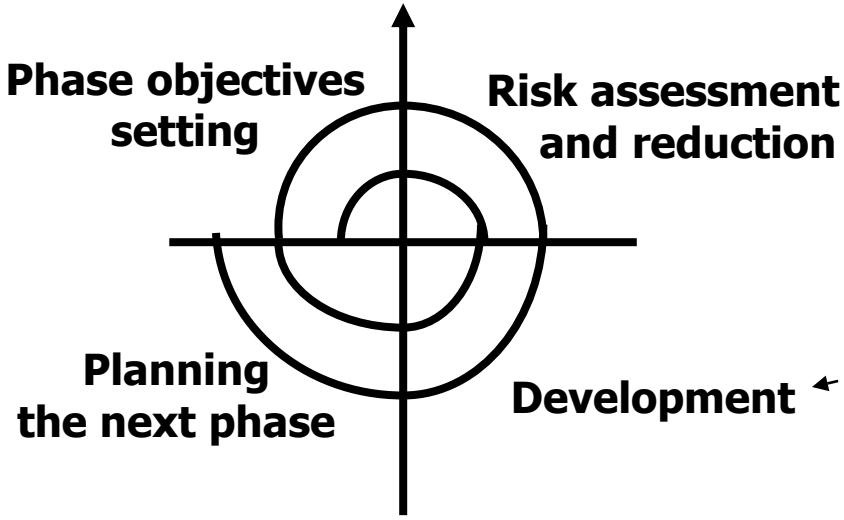
Process iteration – the incremental model

- In the figure above it is assumed that each increment follows a linear model (waterfall). In reality, each iteration (increment) can incorporate any appropriate basic development paradigm [Som01].
- Advantages of incremental development [Som07,Som16,JBR99]:
  - It provides a framework in which requirements can be handled efficiently, and customers do not have to wait until the entire system is delivered to gain value from it.
  - The risks are reduced to the cost of a single increment.
- The main problem (challenge) in the incremental development [Som01,Som07]:
  - It may be difficult to map the customer's requirements onto increments of the appropriate size.

Process iteration – spiral development

In the **spiral model** [Boe88] (see also [Som07, Pre15]) software is developed in a series of incremental releases, and the process is represented as an evolutionary spiral. **Risks** are considered in each phase (loop) of the spiral.

- Each phase (loop) in the spiral is split into a number of task regions:

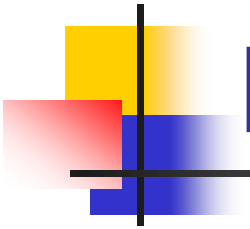


Prototypes may be used as a risk reduction mechanism.

Identified risks help with selecting the most appropriate basic development paradigm for each phase.

Process iteration – concurrent engineering

- The **concurrent development** model [DP94,She94,Pre15] provides an accurate description of the current state of a project, which is represented as a network of concurrent and communicating activities.
  - An event (e.g. a late requirements change) within a given activity (e.g. **analysis**) can trigger transitions among the states of another (concurrent) activity (e.g. the **design** activity moves from the under development state into the awaiting changes state).
- **State transition diagrams** are usually employed to represent activities in the concurrent development model. This model - sometimes called **concurrent engineering** – depicts the software process as a collection of communicating automata.



References

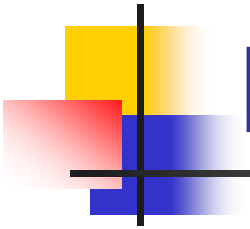
[BK08] C. Baier, J.-K. Katoen. Principles of Model Checking. The MIT Press Cambridge, 2008.

[Bec99] K. Beck. Embracing Change with Extreme Programming. *IEEE Computer* 32(10):70-78, 1999.

[Boe88] B. Boehm. A spiral model of software development and enhancement. *IEEE Computer*, 21(5):61-72, 1988.

[CHVB18] E.M. Clarke, T.A. Henzinger, H. Veith and R. Bloem, Editors, *Handbook of Model Checking*, Springer, 2018.

[Crr99] E. Currie. *The Essence of Z*. Prentice Hall, 1999.



References

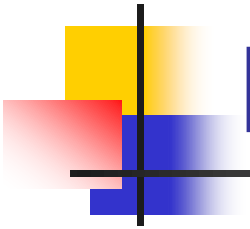
[DP94] A. Davis, P. Pitaram. A concurrent process model for software development. *Software Engineering Notes*, ACM Press, 19(2):38-51, 1994.

[EM85] H. Ehrig and B. Mahr. *Fundamentals of Algebraic Specification, part 1*. Springer, 1985.

[EM90] H. Ehrig and B. Mahr. *Fundamentals of Algebraic Specification, part 2*. Springer, 1990.

[Jac97] J. Jacky. *The Way of Z: Practical Programming with Formal Methods*. Cambridge University Press, 1997.

[JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.



References

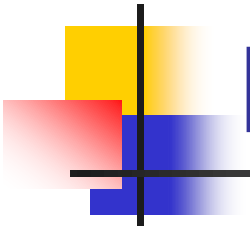
[Kru03] P. Kruchten. *The Rational Unified Process: An Introduction* (3rd edition). Addison-Wesley, 2003.

[Pre97] R. Pressman. *Software Engineering: A Practitioner's Approach*, (4th edition). McGraw-Hill, 1997.

[Pre10] R. Pressman. *Software Engineering: A Practitioner's Approach*, (7th edition). McGraw-Hill, 2010.

[Pre15] R. Pressman. *Software Engineering: A Practitioner's Approach*, (8th edition). McGraw-Hill, 2015.

[Roy70] W. Royce. Managing the development of large software systems: concepts and techniques. In *Proc. IEEE WESTCON*, 1970.



References

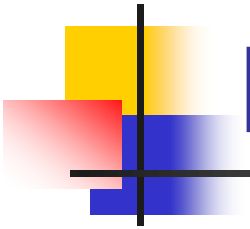
[She94] W. Sheleg. Concurrent Engineering: A New Paradigm for C/S Development. *Application Development Trends*, 1(6):28-33, 1994.

[Spi92] J.M. Spivey. *The Z Notation: A Reference Manual*. Prentice-Hall, 1992.

[Som89] I. Sommerville. *Software Engineering,* (3rd edition). Addison-Wesley, 1989.

[Som01] I. Sommerville. *Software Engineering,* (6th edition). Addison-Wesley, 2001.

[Som05] I. Sommerville. *Software Engineering,* (7th edition). Addison-Wesley, 2005.



References

[Som07] I. Sommerville. *Software Engineering,* (8th edition). Addison-Wesley, 2007.

[Som11] I. Sommerville. *Software Engineering,* (9th edition). Addison-Wesley, 2011.

[Som16] I. Sommerville. *Software Engineering,* (10th edition). Addison-Wesley, 2016.